



**SCHOOL OF
COMPUTING**

Design Analysis of Algorithm

CSE211

Week-5

CH.SC.U4CSE24221

TEJESH KANTA

Merge Sort

Code:

```
C mergesort.c > ...
1  #include <stdio.h>
2
3
4  void merge(int arr[], int left, int mid, int right)
5  {
6      int i, j, k;
7      int n1 = mid - left + 1;
8      int n2 = right - mid;
9
10     int L[n1], R[n2];
11
12     for (i = 0; i < n1; i++)
13         L[i] = arr[left + i];
14     for (j = 0; j < n2; j++)
15         R[j] = arr[mid + 1 + j];
16
17     i = 0;
18     j = 0;
19     k = left;
20
21
22     while (i < n1 && j < n2)
23     {
24         if (L[i] <= R[j])
25             arr[k++] = L[i++];
26         else
27             arr[k++] = R[j++];
28     }
29
30
31     while (i < n1)
32         arr[k++] = L[i++];
33
34     while (j < n2)
35         arr[k++] = R[j++];
36 }
```

```

37
38
39 void mergeSort(int arr[], int left, int right)
40 {
41     if (left < right)
42     {
43         int mid = (left + right) / 2;
44
45         mergeSort(arr, left, mid);
46         mergeSort(arr, mid + 1, right);
47         merge(arr, left, mid, right);
48     }
49 }
50
51
52 int main()
53 {
54     int arr[100], n, i;
55
56     printf("Enter number of elements: ");
57     scanf("%d", &n);
58
59     printf("Enter array elements:\n");
60     for (i = 0; i < n; i++)
61         scanf("%d", &arr[i]);
62
63     mergeSort(arr, 0, n - 1);
64
65     printf("Sorted array:\n");
66     for (i = 0; i < n; i++)
67         printf("%d ", arr[i]);
68
69     return 0;
70 }
71

```

Output:

```

[kantatejesh@KANTAs-MacBook-Air Website % gcc mergesort.c -o mergesort
[kantatejesh@KANTAs-MacBook-Air Website % ./mergesort
Enter number of elements: 10
Enter array elements:
23 34 10 31 22 29 47 56 67 56
Sorted array:
10 22 23 29 31 34 47 56 56 67 %
kantatejesh@KANTAs-MacBook-Air Website %

```

Space Complexity and its justification:

- Time Complexity (All cases): $O(n \log n)$
- Reason: Array is divided into halves recursively $\rightarrow \log n$ levels
- Merge Cost: At each level, n elements are merged $\rightarrow O(n)$ work
- Space Complexity: $O(n)$
- Reason: Temporary array $b[]$ is used during merging (recursion stack $O(\log n)$ is smaller)

Quick Sort

Code:

```
1  #include <stdio.h>
2
3  void swap(int *a, int *b)
4  {
5      int temp = *a;
6      *a = *b;
7      *b = temp;
8  }
9
10
11 int partition(int arr[], int low, int high)
12 {
13     int pivot = arr[high];
14     int i = low - 1;
15
16     for (int j = low; j < high; j++)
17     {
18         if (arr[j] < pivot)
19         {
20             i++;
21             swap(&arr[i], &arr[j]);
22         }
23     }
24     swap(&arr[i + 1], &arr[high]);
25     return i + 1;
26 }
27
28
29 void quickSort(int arr[], int low, int high)
30 {
31     if (low < high)
32     {
33         int pi = partition(arr, low, high);
34
35         quickSort(arr, low, pi - 1);
36         quickSort(arr, pi + 1, high);
37     }
```

```
37     }
38 }
39
40
41 int main()
42 {
43     int arr[100], n;
44
45     printf("Enter number of elements: ");
46     scanf("%d", &n);
47
48     printf("Enter array elements:\n");
49     for (int i = 0; i < n; i++)
50         scanf("%d", &arr[i]);
51
52     quickSort(arr, 0, n - 1);
53
54     printf("Sorted array:\n");
55     for (int i = 0; i < n; i++)
56         printf("%d ", arr[i]);
57
58     return 0;
59 }
60
61
```

Output:

```
[kantatejesh@KANTAs-MacBook-Air ~ % cd Desktop
[kantatejesh@KANTAs-MacBook-Air Desktop % cd Website
[kantatejesh@KANTAs-MacBook-Air Website % gcc quicksort.c -o quicksort
[kantatejesh@KANTAs-MacBook-Air Website % ./quicksort
Enter number of elements: 7
Enter array elements:
12 34 56 23 31 10 8
Sorted array:
8 10 12 23 31 34 56 %
```

Space Complexity and its justification:

- ✓ Best & Average Time Complexity: $O(n \log n)$
- ✓ Worst Time Complexity: $O(n^2)$ (when pivot is smallest/largest → unbalanced partitions)
- ✓ Reason: Partitioning takes $O(n)$ and recursion depth is $\log n$ (best/avg) or n (worst)
- ✓ Space Complexity (Average): $O(\log n)$ due to recursion stack
- ✓ Space Complexity (Worst): $O(n)$ when recursion becomes linear